TECHNICAL DOCUMENTATION

Engagement Scoring & Lead Pipeline System

BizCivitas Performance Marketing

VERSION	DATE	PLATFORM	STATUS
1.0	March 16, 2026	Next.js + Express + MongoDB	Production Ready

CONTENTS

Table of Contents

01	System Overview	Goals, Architecture, Tech Stack
02	Pipeline Stages	New → Cold → Warm → Hot → Converted
03	Engagement Scoring Model	Events, Scores, Categories
04	Database Structure	PmScoringConfig, PmInquiry, Activity Log
05	Frontend Tracking	Hook, Events, SessionStorage
06	Backend Processing	Controllers, Score Calculation, Pipeline Assignment
07	Complete Data Flow	End-to-End Journey
08	API Reference	All Endpoints
09	Admin Panel	Scoring Configuration Page
10	Score Decay & Maintenance	Inactive Lead Handling
11	Example Scenarios	User Journeys
12	File Structure	All Files Created/Modified

SECTION 01

System Overview

An automated lead intelligence system that tracks visitor behavior and assigns pipeline stages dynamically.

What It Does

This system tracks **user engagement on the BizCivitas performance marketing website** and dynamically assigns an **engagement score** based on user behavior. Based on the score, users are automatically categorized into a pipeline stage.

> **Key Principle:** The "Converted" stage is determined by **payment/membership data**, not by engagement score. Payment always overrides score-based stages.

System Goals

- Track user engagement across the website in real-time
- Assign scores based on user actions (page visits, scroll depth, form interactions, etc.)
- Automatically update lead pipeline stage based on accumulated score
- Identify highly interested users before they convert
- Detect converted users via Razorpay payment records
- Allow admin to customize scoring values and pipeline thresholds
- Support custom event categories added by admin
- Maintain real-time engagement analytics in admin panel

Tech Stack

Frontend

Next.js 14 (App Router), TypeScript, Tailwind CSS, Lucide Icons

Backend

Express.js, MongoDB (Mongoose ODM), Node.js

Payment

Admin

Razorpay (order creation, checkout, signature verification)

Next.js admin panel with JWT httpOnly cookie auth

SECTION 02

Pipeline Stages

Users automatically flow through 5 stages based on their engagement score.

NEW	COLD	WARM	HOT	CONVERTED

STAGE	SCORE RANGE	DESCRIPTION	TRIGGER
NEW	0 – 10	First interaction with the platform	Automatic (default stage)
COLD	11 – 25	Low engagement visitor, browsing	Score crosses cold threshold
WARM	26 – 50	Interested visitor, exploring content	Score crosses warm threshold
HOT	51+	Highly engaged potential lead	Score crosses hot threshold
CONVERTED	Payment detected	User has purchased membership	Razorpay payment verified

Important Rule: Converted users override ALL engagement scores. Once payment is verified, the pipeline stage becomes "Converted" regardless of the score value.

Pipeline Assignment Logic

```
function getPipelineStage(score, isConverted) {
  if (isConverted) return "converted";

  const thresholds = await getThresholdsFromDB();  // { cold: 11, warm: 26, hot: 51

  if (score < thresholds.cold)  return "new";    // 0-10
  if (score < thresholds.warm)  return "cold";   // 11-25
  if (score < thresholds.hot)   return "warm";   // 26-50
```

```
    return "hot";                                   // 51+
  }
```

> **Thresholds are configurable** from the admin panel at `/admin/scoring`. The values (11, 26, 51) are defaults that can be adjusted per business needs.

SECTION 03

Engagement Scoring Model

User engagement is measured through behavioral events, each assigned a point value.

System Events BUILT-IN

These 11 events are tracked automatically by the frontend. Their scores can be adjusted but they cannot be deleted.

EVENT	KEY	SCORE	HOW IT'S TRIGGERED
Page Visit	`page_visit`	+2	Every page the user loads
Return Visit	`return_visit`	+5	User returns (localStorage check)
Stay 30 seconds	`time_30sec`	+3	Timer fires after 30s on page
Stay 2 minutes	`time_2min`	+8	Timer fires after 120s on page
Scroll 50%	`scroll_50`	+3	Scroll listener detects 50%+ depth
CTA Click	`cta_click`	+5	User clicks a call-to-action button
Form Started	`form_started`	+8	First focus on any form input
Form Submitted	`form_submitted`	+20	Inquiry form submit
File Download	`file_download`	+10	User downloads a brochure/file
Pricing Page	`pricing_visit`	+15	User visits /checkout or pricing page

EVENT	KEY	SCORE	HOW IT'S TRIGGERED
5+ Pages Visited	`multiple_pages`	+10	Unique page count exceeds 5

Custom Events ADMIN CREATED

Admins can create additional scoring events from the admin panel. Custom events have the same structure but can be fully edited and deleted.

> **Example custom events:** "Video Watched" (+12), "Testimonial Clicked" (+7), "Brochure Downloaded" (+10), "Chat Initiated" (+8)

SECTION 04

Database Structure

MongoDB collections that power the engagement scoring system.

1. PmScoringConfig Collection (Singleton — 1 document)

Stores all scoring configuration. Only one document exists in this collection.

```
{
  "events": [
    {
      "_id": "ObjectId",
      "key": "page_visit",
      "label": "Page Visit",
      "description": "Each page the user visits",
      "score": 2,
      "icon": "Eye",
      "isSystem": true,
      "isActive": true
    },
    // ... 11 system events + any custom events
  ],
  "thresholds": { "cold": 11, "warm": 26, "hot": 51 },
  "decay": {
    "enabled": true,
    "inactiveDays": 30,
    "decayAmount": 10
  }
}
```

2. PmInquiry Collection (Per Lead)

Each inquiry/lead has engagement tracking fields alongside their contact information.

```
{
  "fullName": "John Doe",
  "email": "john@email.com",
  "phone": "9876543210",
  "companyName": "Acme Corp",
```

```
    "status": "warm",
    "engagementScore": 38,
    "pipelineStage": "warm",
    "lastActivity": "form_submitted",
    "activityLog": [
      { "event": "page_visit", "scoreAdded": 2, "timestamp": "2026-03-16T10:00:00Z" },
      { "event": "time_2min", "scoreAdded": 8, "timestamp": "2026-03-16T10:02:00Z" },
      { "event": "pricing_visit", "scoreAdded": 15, "timestamp": "2026-03-16T10:03:00Z" },
      { "event": "form_submitted", "scoreAdded": 20, "timestamp": "2026-03-16T10:05:00Z" }
    ]
  }
```

SECTION 05

Frontend Tracking

How user behavior is captured on the website before and after inquiry creation.

Tracking Hook: `useEngagementTracker`

A React hook that automatically tracks all user behavior. It runs on every page and stores events in `sessionStorage` until the inquiry is created.

Auto-Tracked Events (no code needed per page)

Page Visit
Fires on every page load

Return Visit
Checks localStorage for prior visit

Time on Page
Interval checks at 30s and 2min

Scroll Depth
Scroll listener for 50%+ depth

Pricing Visit
Detects /checkout or /pricing URLs

Multiple Pages
Tracks unique page count > 5

Interaction-Triggered Events

EVENT	WHERE IT'S TRACKED	HOW
`cta_click`	Any CTA button	`trackEvent("cta_click")`
`form_started`	InquiryModal	`onFocus` on form (first time only)
`form_submitted`	InquiryModal	After successful form submit

Storage Strategy

Before inquiry exists: All events are stored in `sessionStorage` as an array. No API calls are made to avoid creating orphan data.

> **After form submit:** The `syncEventsToInquiry(inquiryId)` function sends ALL accumulated events to the backend in a single batch API call. Then sessionStorage is cleared.

Session ID

A unique session ID is generated per browser session and stored in `sessionStorage`. Format: `ses_1710590400000_a7b3c9d`

SECTION 06

Backend Processing

How the backend receives events, calculates scores, and assigns pipeline stages.

Score Calculation

The backend reads scoring values from the `PmScoringConfig` collection (not hardcoded). This allows admin to change scores without code deployment.

```
// Build score map from DB config
const getScoreMap = async () => {
  const config = await PmScoringConfig.findOne();
  const map = {};
  for (const e of config.events) {
    if (e.isActive) map[e.key] = e.score;
  }
  return map;  // { page_visit: 2, return_visit: 5, ... }
};
```

Sync Process (POST /engagement/sync)

1. Receive `inquiryId` + `events[]` array from frontend
2. Fetch `PmScoringConfig` from DB to get current score values
3. Loop through each event, look up score from config
4. Add each event to `inquiry.activityLog`
5. Accumulate `inquiry.engagementScore`
6. Calculate pipeline stage using thresholds from DB
7. Auto-update `inquiry.status` to match pipeline stage
8. Save inquiry document

Status Auto-Update Rules

CURRENT STATUS	NEW PIPELINE STAGE	ACTION
new	cold / warm / hot	Status updates to match

CURRENT STATUS	NEW PIPELINE STAGE	ACTION
contacted	any	No auto-update (manual override)
converted	any	Never downgraded

SECTION 07

Complete Data Flow

End-to-end journey from user's first visit to conversion.

Step-by-Step Flow

1. **User visits landing page**
   EngagementTracker hook activates. Tracks page_visit (+2). Checks for return visit. Starts time and scroll listeners.

2. **User browses and engages**
   Events accumulate in sessionStorage: time_30sec (+3), scroll_50 (+3), time_2min (+8). Score builds up locally.

3. **User opens inquiry form**
   form_started (+8) tracked on first input focus. Events still in sessionStorage only.

4. **User submits form**
   POST /inquiry/add creates PmInquiry. form_submitted (+20) added. syncEventsToInquiry() sends ALL events to backend. Score calculated, pipeline assigned.

5. **Redirect to checkout**
   pricing_visit (+15) tracked on checkout page. Score updates via real-time track API.

6. **User completes payment**
   Razorpay payment verified. Status → "converted". Pipeline → "converted". User account created. Welcome email sent.

Example Score Accumulation

Visit page +2 → Stay 2min +8 → Scroll 50% +3 → Form start +8 → Submit +20 → Pricing +15 = **56 pts** HOT

SECTION 08

API Reference

All endpoints for the engagement scoring system.

Engagement Tracking

METHOD	ENDPOINT	DESCRIPTION	ACCESS
POST	`/api/v1/pm/engagement/track`	Track single event	Public
POST	`/api/v1/pm/engagement/sync`	Batch sync events to inquiry	Public
GET	`/api/v1/pm/engagement/:id`	Get engagement data	Admin
POST	`/api/v1/pm/engagement/decay`	Run score decay	Admin

Scoring Configuration

METHOD	ENDPOINT	DESCRIPTION	ACCESS
GET	`/api/v1/pm/scoring-config`	Get scoring config	Admin
PATCH	`/api/v1/pm/scoring-config`	Update scores/thresholds/decay	Admin
POST	`/api/v1/pm/scoring-config/reset`	Reset to defaults	Admin
POST	`/api/v1/pm/scoring-config/events`	Add custom event	Admin
PATCH	`/api/v1/pm/scoring-config/events/:id`	Edit event	Admin
DELETE	`/api/v1/pm/scoring-config/events/:id`	Delete custom event	Admin

Key Request/Response Examples

POST /engagement/sync

```
// Request
{
  "inquiryId": "507f1f77bcf86cd799439011",
  "events": [
    { "event": "page_visit", "timestamp": "2026-03-16T10:00:00Z" },
    { "event": "time_2min", "timestamp": "2026-03-16T10:02:00Z" },
    { "event": "form_submitted", "timestamp": "2026-03-16T10:05:00Z" }
  ]
}

// Response
{
  "statusCode": 200,
  "data": {
    "engagementScore": 30,
    "pipelineStage": "warm",
    "eventsProcessed": 3
  }
}
```

SECTION 09

Admin Panel

The scoring configuration page at `/admin/scoring`.

Features

- **System Events** — View all 11 built-in events with toggle on/off and editable scores
- **Custom Events** — Full CRUD: Add, Edit, Delete custom scoring events
- **Pipeline Thresholds** — Visual bar showing stage ranges + editable threshold inputs
- **Score Decay** — Toggle, configure inactive days and decay amount
- **Example Journey** — Live preview of score calculation with current values
- **Save / Reset** — Save changes to DB or reset everything to defaults

Inquiries Page Enhancements

The `/admin/inquiries` page now shows:

- **Score column** — Color-coded badge showing engagement score per lead
- **Pipeline column** — Badge showing current pipeline stage (New/Cold/Warm/Hot)
- **Detail modal** — Shows engagement score, pipeline stage, and last activity
- **Status filter** — Updated with Cold, Warm, Hot filter options

Admin Sidebar Navigation

MENU ITEM	PATH	DESCRIPTION
Dashboard	`/admin`	Overview stats
Inquiries	`/admin/inquiries`	Lead management with scores
Plans	`/admin/plans`	Membership plan CRUD
Members	`/admin/members`	Converted leads
Scoring	`/admin/scoring`	Scoring configuration

SECTION 10

Score Decay & Maintenance

Preventing inactive users from staying in Hot stage forever.

How Score Decay Works

SETTING	DEFAULT	DESCRIPTION
Enabled	Yes	Score decay is active
Inactive Days	30	Days without activity before decay triggers
Decay Amount	10	Points subtracted per decay cycle

Decay Logic

```
// Finds all non-converted inquiries inactive for 30+ days
// Reduces their score by 10 (minimum 0)
// Recalculates pipeline stage

Example:
  Hot lead (score: 55) inactive 30 days
  → Score becomes 45
  → Pipeline: Hot → Warm

  Warm lead (score: 30) inactive 30 days
  → Score becomes 20
  → Pipeline: Warm → Cold
```

> **Converted leads are never decayed.** Once a user pays, their status remains "Converted" permanently regardless of activity or score.

Running Decay

Score decay can be triggered via:

- **API call:** `POST /api/v1/pm/engagement/decay`

- **Cron job:** Set up a daily/weekly cron to call the decay endpoint
- **Manual:** Can be triggered from admin panel in future

SECTION 11

Example Scenarios

Real-world user journeys and their resulting pipeline stages.

Scenario 1: Quick Bounce

Visit +2 = **2 pts** NEW

User lands on homepage and leaves quickly. Minimal engagement.

Scenario 2: Casual Browser

Visit +2 → 30 sec +3 → Scroll +3 → 2nd page +2 = **10 pts** NEW

User browses a couple of pages, reads some content, but doesn't engage deeply.

Scenario 3: Interested Visitor

Visit +2 → 2 min +8 → Scroll +3 → CTA +5 = **18 pts** COLD

User reads content for 2 minutes and clicks a CTA. Showing initial interest.

Scenario 4: Engaged Lead

Return +5 → 2 min +8 → Pricing +15 → Form start +8 = **36 pts** WARM

Returning visitor who checks pricing and starts filling the form. Strong buying signal.

Scenario 5: Full Conversion

All above +36 → Submit +20 → Payment = **56 pts** CONVERTED

User completes the full journey: browse → engage → submit → pay. Score was "Hot" (56) but payment overrides to "Converted".

SECTION 12

File Structure

All files created and modified for this system.

Backend (bizcivitas-backend)

FILE	ACTION	PURPOSE
`models/performance-marketing/pmInquiry.model.js`	MODIFIED	Added engagementScore, pipelineStage, activityLog fields
`models/performance-marketing/pmScoringConfig.model.js`	CREATED	Scoring config with events array, thresholds, decay
`controllers/performance-marketing/pmEngagement.controller.js`	CREATED	Track, sync, decay, get engagement data
`controllers/performance-marketing/pmScoringConfig.controller.js`	CREATED	CRUD for scoring config and events
`routes/performance-marketing/pmEngagement.routes.js`	CREATED	Engagement tracking routes
`routes/performance-marketing/pmScoringConfig.routes.js`	CREATED	Scoring config routes
`app.js`	MODIFIED	Mounted engagement + scoring-config routes

Frontend (bizcivitas-performance-marketing)

FILE	ACTION	PURPOSE
`src/hooks/useEngagementTracker.ts`	CREATED	React hook for tracking all user behavior

FILE	ACTION	PURPOSE
`src/components/EngagementTracker.tsx`	CREATED	Client component wrapper for the hook
`src/app/admin/scoring/page.tsx`	CREATED	Admin scoring configuration page
`src/app/page.tsx`	MODIFIED	Added EngagementTracker to landing page
`src/app/checkout/page.tsx`	MODIFIED	Added tracking hook for pricing_visit
`src/components/InquiryModal.tsx`	MODIFIED	Added form_started, form_submitted, syncEvents
`src/components/admin/AdminShell.tsx`	MODIFIED	Added Scoring nav item
`src/app/admin/inquiries/page.tsx`	MODIFIED	Added Score & Pipeline columns

BizCivitas Performance Marketing • Engagement Scoring System v1.0 • March 2026